

Cahier de TP

« Apache Solr »

Pré-requis :

- OS recommandé : Linux ou Windows 10
- JDK17+
- Bon éditeur XML (Notepad++, Vscodé, ...)
 - Plusieurs machines en réseau ou Docker pour le TP Optionnel sur SolrCloud

Table des matières

Atelier 1 : Installation et démo.....	2
Atelier 2 : Mise en place de cœur.....	3
2.1 Création de cœur.....	3
2.2 Propriétés de configuration.....	4
2.3 Comprendre l'analyse.....	5
2.4 Analyseur phonétique français.....	5
Atelier 3 : Indexation.....	6
3.1 Indexation XML.....	6
3.2 Indexation JSON.....	6
3.3 Indexation CSV (Optionnel).....	6
Atelier 4 : Importation de documents bureautique.....	7
Atelier 5 : Importation base de données.....	9
Atelier 6 : Configuration request handler et 1ères recherches.....	10
6.1 Configuration.....	10
6.2 Requêtes.....	10
Atelier 7 : Différents types de recherches.....	11
7.1 Spell-check.....	11
1.1 Ajouter le composant (DirectSolrSpellChecker).....	11
7.2 Highlight.....	12
Atelier 8 : Agrégation de documents.....	13
8.1 Facettes.....	13
Atelier 9 : Recherche géo-graphique et client Java.....	14
Atelier 10 : SolrCloud.....	15

Atelier 1 : Installation et démo

Récupérer une distribution de Apache Solr et la dézipper dans un répertoire de travail : (\$SOLR_HOME)

Démarrer la configuration cloud

```
cd $SOLR_HOME
./bin/solr start -e cloud
```

Répondre à l'assistant en choisissant :

- 2 nœuds
- les ports proposés
- une collection de données nommée **techproducts**
- 2 shards, 2 répliques
- **sample_techproducts_configs** comme configuration

Ouvrir et découvrir l'interface d'administration : <http://localhost:8983/solr/>

En particulier visualiser le lien cloud

Indexer des données avec :

```
./bin/post -c techproducts example/exampledocs/*
```

OU

```
./bin/solr post -c techproducts example/exampledocs/*
```

Ensuite visualiser les documents via l'interface d'admin en effectuant des recherches

- Tous les documents
- Les documents contenant le terme **foundation**
- Les documents dont le champ **cat** contient **electronics**
- Les documents contenant la phrase « **Memory stick** »

Vous pouvez également utiliser curl pour effectuer ses requêtes

Supprimer la collection via :

```
bin/solr delete -c techproducts
```

Arrêter le cluster

```
bin/solr stop -all
```

Atelier 2 : Mise en place de cœur

2.1 Création de cœurs

Démarrer une configuration standalone de Solr

```
bin/solr start
```

Vérifier <http://localhost:8983/solr>

Schéma géré par Solr, schemaless

Créer un cœur nommé *formation_managed*

```
bin/solr create -c formation_managed -d _default
```

Visualiser les fichiers de configuration créés dans `server/solr/formation_managed/`.

En particulier :

- Le fichier `managed-schema`
- `solrconfig.xml` et la balise `schemaFactory`

Visualiser la config via :

- L'API Rest :

```
curl "http://localhost:8983/solr/formation_managed/config"
```

- Via la console d'administration

Créer un autre cœur nommé *formation*, en créant au préalable un répertoire de configuration, en se positionnant en mode contrôle exclusif du schéma sans possibilité d'ajout de champ

Créer le répertoire et copier la configuration de base :

```
cp -r server/solr/configsets/_default server/solr/formation
```

Remplacer la config par défaut en éditant `solrconfig.xml` et en fixant la propriété `schemaFactory` à ***ClassicIndexSchemaFactory***

```
<schemaFactory class="ClassicIndexSchemaFactory"/>
```

Renommer le fichier *managed-schema* en *schema.xml*

Si SolrCloud

```
bin/solr zk upconfig -z <zk_host>:2181 -n mon_configset -d /opt/solr/configs/mon_configset
```

Créer le cœur formation :

```
bin/solr create -c formation -d server/solr/formation
```

Visualiser la config via :

- L'API Rest :

```
curl "http://localhost:8983/solr/formation/config"
```

- Via la console d'administration

Test de l'indexation, essayer ces 2 requêtes d'indexation

```
curl "http://localhost:8983/solr/formation_managed/update?
commit=true" \
-H "Content-Type: application/json" \
-d ' [{"id": "1", "nom": "PLB", "ville": "Paris"} ] '
```

ET

```
curl "http://localhost:8983/solr/formation/update?commit=true" \
-H "Content-Type: application/json" \
-d ' [{"id": "1", "nom": "PLB", "ville": "Paris"} ] '
```

2.2 Propriétés de configuration

Créer un fichier *core.properties* dans le répertoire du cœur formation, y positionner une propriété, recharger le cœur et vérifier.

Constater l'absence (ou l'état) de l'overlay

```
curl "http://localhost:8983/solr/formation/config/overlay?
indent=on"
```

Positionner la propriété `update.autoCreateFields` de cette façon :

```
bin/solr config -c formation -p 8983 \
-action set-user-property \
-property update.autoCreateFields \
-value false
```

Vérifier l'overlay :

```
# Vérifier l'overlay côté API
```

```
curl "http://localhost:8983/solr/formation/config/overlay?indent=on"
```

```
# Vérifier côté disque
```

```
cat server/solr/formation/conf/configoverlay.json
```

Et la configuration effective du coeur

2.3 Schéma

Visualiser les schémas de 2 coeurs via l'API :

```
curl "http://localhost:8983/solr/formation\_managed/schema?indent=on"
```

```
curl "http://localhost:8983/solr/formation/schema?indent=on"
```

Identifier les différences

Explorer les types de champs et champs dynamiques :

```
curl "http://localhost:8983/solr/formation_managed/schema/fieldtypes?indent=on"
```

```
curl "http://localhost:8983/solr/formation/schema/fieldtypes?indent=on"
```

Ateliers 3 : Analyse, Indexation

3.1 Comprendre l'analyse

Testez l'analyse sur la chaîne « Une formation débutant sur SolR » :

- Avec le type de champ ***text_ws***
- Avec le type de champ ***text_general***
- Avec le type de champ ***text_fr***

Que constatez-vous ?

Lisez les commentaires associés à ces types de champs dans ***schema.xml***

Essayez avec :

« *Overview of Documents, Fields, and Schema Design* » et le champ phonétique ***english***

Exécuter le test fourni pour visualiser les effets des annotations

3.2 Analyseur phonétique français

Définir un nouveau type de champs effectuant une analyse phonétique en français, y associer un champ dynamique et tester l'analyse

3.3 Indexation XML

Visualiser le fichier XML fourni et dans le cœur *formation* modifier le fichier ***schema.xml*** pour être le plus précis sur les champs utilisés.

Effectuer la bonne configuration dans ***solrconfig.xml*** ou ***configoverlay.json*** pour interdire tout nouveau champ dans le schéma

Utiliser le fichier XML fourni pour alimenter les 2 cœurs (*formation* et *formation_managed*)

3.4 Indexation JSON

Reprendre le fichier ***slides.json*** fourni et effectuer une requête permettant d'insérer le document dans le schéma maîtrisé précédent

3.5 Indexation CSV (Optionnel)

Ajouter un document au format CSV dans les cœurs précédents

Atelier 4 : Importation de documents bureautique

Créer un nouveau cœur

```
bin/solr create -c docs-bureau -n _default
```

S'assurer d'avoir des champs simples pour capturer le titre, le nom du fichier, le type MIME, et le texte intégral.

Éventuellement ajouter des champs via l'API. Par exemple :

```
curl -sX POST -H 'Content-type:application/json' \  
  http://localhost:8983/solr/docs-bureau/schema \  
  -d '{  
    "add-field": {  
      "name": "file_path",  
      "type": "string",  
      "stored": true  
    }  
  }'  
  
# Champ pour titre  
curl -sX POST -H 'Content-type:application/json' \  
  http://localhost:8983/solr/docs-bureau/schema \  
  -d '{  
    "add-field": {  
      "name": "title",  
      "type": "text_general",  
      "stored": true  
    }  
  }'  
  
# Champ pour type MIME  
curl -sX POST -H 'Content-type:application/json' \  
  http://localhost:8983/solr/docs-bureau/schema \  
  -d '{  
    "add-field": {  
      "name": "content_type",  
      "type": "string",  
      "stored": true  
    }  
  }'
```

Éventuellement, copier le texte intégral dans le champ de recherche global

```
curl -sX POST -H 'Content-type:application/json' \  
  http://localhost:8983/solr/docs-bureau/schema \  
  -d '{  
    "add-field": {  
      "name": "text_integral",  
      "type": "text_general",  
      "stored": true  
    }  
  }'
```

```
http://localhost:8983/solr/docs-bureau/schema \
-d '{
  "add-copy-field": {
    "source": "content_txt",
    "dest": "_text_"
  }
}'
```

Configurer le gestionnaire d'importation

```
<!-- À insérer dans solrconfig.xml -->
<requestHandler name="/update/extract"
class="solr.extraction.ExtractingRequestHandler">
  <lst name="defaults">
    <str name="lowernames">true</str>
    <str name="uprefix">ignored_</str>
    <str name="captureAttr">true</str>
    <str name="fmap.content">content_txt</str>
    <str name="fmap.author">author</str>
    <str name="commit">true</str>
  </lst>
</requestHandler>
```

Utiliser l'utilitaire *solr* pour indexer les documents bureautiques fournis.

```
bin/post -c docs-bureau ~/docs
```


Atelier 5 : Importation base de données

```
# 1) Démarrer Solr avec le package manager activé
bin/solr start -c -Denable.packages=true

# 2) Ajouter le dépôt du plugin DIH
bin/solr package add-repo data-import-handler
"https://raw.githubusercontent.com/SearchScale/dataimporthandler/master/repo/"

# 3) Installer le package
bin/solr package install data-import-handler

# 4) Créer ta collection (ou core)
bin/solr create -c db-demo -n _default

# 5) Déployer le requestHandler /dataimport sur la collection
bin/solr package deploy data-import-handler -y -collections db-demo
```

data-config.xml

```
<dataConfig>
  <dataSource type="JdbcDataSource"
    driver="org.postgresql.Driver"
    url="jdbc:postgresql://localhost:5432/shop"
    user="plb" password="pass"/>
  <document>
    <entity name="product" query="SELECT id, name, category,
price, description, updated_at FROM products">
      <field column="id" name="id"/>
      <field column="name" name="name"/>
      <field column="category" name="category"/>
      <field column="price" name="price"/>
      <field column="description" name="description"/>
      <field column="updated_at" name="updated_at"/>
    </entity>
  </document>
</dataConfig>
```

Atelier 6 : Configuration request handler et 1ères recherches

6.1 Configuration

Base bureautique :

Configurer le request handler pour :

- Travailler par défaut sur le champ *content*
- Forcer une sortie en *json*
- Utiliser par défaut le parseur lucene
- Désactiver les *searchcomponent* : *facet*, *morelikethis* et *highlight*
- Ajouter *_score* dans les champs retournés
- Informations de debug

6.2 Requêtes

Effectuer les recherches suivantes en utilisant la syntaxe lucene :

- Documents répondant à « Java »
- Documents ne répondant pas à « Java »
- Limiter les documents retournés de la première requête
- Documents dont le contenu répond à « Java »
- Documents PDF dont le contenu répond à « Java »
- Documents dont le contenu répond à « SolR»•
- Documents dont le champ titre contient administration
- Document créés après une date particulière
- Document créés après une date particulière et dont le contenu répondant « Java Elastic Search » mais pas « Administration »

Atelier 7 : Différents types de recherches

7.1 Spell-check

2 étapes

- Déclarer le composant **spellcheck**.
- L'ajouter à `/select` via `last-components` (ou créer un handler dédié).
- Redémarrer et forcer le build

Choisir les champs appropriés pour faire du spell-checking, author

```
<searchComponent name="spellcheck"
class="solr.SpellCheckComponent">
  <lst name="spellchecker">
    <str name="name">default</str>
    <str name="classname">solr.DirectSolrSpellChecker</str>
    <str name="field">author</str>
    <float name="accuracy">0.5</float>
    <int name="maxEdits">2</int>
    <int name="minPrefix">1</int>
    <int name="minQueryLength">4</int>
    <int name="maxInspections">5</int>
  </lst>
</searchComponent>
```

Ajouter dans le requestHandler `/select` :

```
<arr name="last-components">
  <str>spellcheck</str>
</arr>
```

Redémarrer puis forcer le build :

```
curl "http://localhost:8983/solr/search-demo/select?
q=:*&rows=0&spellcheck=true&spellcheck.build=true&wt=json"
```

Tester les suggestions, par exemple :

```
curl "http://localhost:8983/solr/search-demo/select?
q=thibEAU&rows=0&spellcheck=true&spellcheck.q=contrcat&spellcheck.
collate=true&wt=json"
```

7.2 Highlight

Utiliser les paramètres de highlight via l'interface d'admin

Atelier 8 : Agrégation de documents

8.1 Facettes

Utiliser les paramètres liés aux facettes

Atelier 9 : Recherche géo-graphique et client Java

Compléter l'application SpringBoot afin de pouvoir importer des données de géo-location dans un cœur SolR :

- Ajout des dépendances vers SolrJ
- Classe de configuration SolR créant un bean HttpClient
- Implémentation d'une classe service ajoutant un document dans un cœur à partir de la classe du modèle *Position.java*

Préparer un cœur définissant des champs géographique avec les types :

- LatLonPointSpatialField
- *SpatialRecursivePrefixTreeFieldType*

Effectuer ensuite les requêtes suivantes :

- Rechercher les documents via un rectangle
- Distance à partir d'un point central
- idem avec en plus des facettes
- Agrégation de type heatmap sur le champ RPT

Atelier 10 : SolrCloud

10.1 Docker

Visualiser le fichier ***docker-compose*** fourni

Démarrer le cluster et vérifier le bon démarrage de tous les processus

Ajouter une collection au cloud avec 2 shards et 1 réplique

10.2 Via tar.gz

Installation Zookeeper :

```
mkdir -p ./zk-ensemble/{data1,data2,data3,logs}
```

```
echo 1 > ./zk-ensemble/data1/myid
```

```
echo 2 > ./zk-ensemble/data2/myid
```

```
echo 3 > ./zk-ensemble/data3/myid
```

Mettre au point conf/zoo1.cfg, conf/zoo2.fg, conf/zoo3.cfg

```
tickTime=2000
```

```
initLimit=10
```

```
syncLimit=5
```

```
dataDir=/home/dthibau/Formations/Solr/MyWork/zk-ensemble/data3
```

```
dataLogDir=/home/dthibau/Formations/Solr/MyWork/zk-ensemble/logs
```

```
clientPort=2183
```

```
# Ports de quorum (communication inter-serveurs : peer : election)
```

```
server.1=127.0.0.1:2888:3888
```

```
server.2=127.0.0.1:2889:3889
```

```
server.3=127.0.0.1:2890:3890
```

```
4lw.commands.whitelist=stat,ruok,svr,conf,mntr
```

Démarrer :

```
bin/zkServer.sh start conf/zoo1.cfg
```

```
bin/zkServer.sh start conf/zoo2.cfg
```

```
bin/zkServer.sh start conf/zoo3.cfg
```

Vérifier :

```
echo srvr | nc 127.0.0.1 2181
echo srvr | nc 127.0.0.1 2182
echo srvr | nc 127.0.0.1 2183
```

Installation Solr en service :

```
VERSION=9.9.0 # adapte à ta version

tar xzf solr-$VERSION.tgz
solr-$VERSION/bin/install_solr_service.sh --strip-components=2
sudo bash ./install_solr_service.sh solr-$VERSION.tgz
```

Éditer /etc/default/solr.in.sh :

```
# Mode cloud

SOLR_MODE=solrcloud

# Ports du ZK ensemble local (+ chroot dédié)
ZK_HOST="127.0.0.1:2181,127.0.0.1:2182,127.0.0.1:2183/solr"

# Port HTTP de ce nœud
SOLR_PORT=8983

# Dossiers (par défaut)
SOLR_HOME="/var/solr/data"
SOLR_LOGS_DIR="/var/solr/logs"
SOLR_PID_DIR="/var/solr"

# Mémoire raisonnable en local
SOLR_JAVA_MEM="-Xms512m -Xmx1g"
```

Crée le **chroot** dans ZooKeeper (une seule fois) :

```
# via zkCli de ZK
```



```
bin/zkCli.sh -server 127.0.0.1:2181 create /solr ""
```

Démarrer le service

```
sudo systemctl enable --now solr
```

```
# UI : http://localhost:8983/solr/#/~cloud
```

Démarrage manuel de la 2ème instance

```
export SOLR_PORT2=7574
```

```
mkdir -p ~/solrcloud-node2/{data,logs,pids}
```

```
sudo -u solr mkdir -p /var/solr2/{data,logs,pids}
```

```
sudo -u solr SOLR_LOGS_DIR="/var/solr2/logs" \
```

```
SOLR_PID_DIR="/var/solr2/pids" \
```

```
/opt/solr/bin/solr start -cloud \
```

```
-p $SOLR_PORT2 \
```

```
-s "/var/solr2/data" \
```

```
-z "127.0.0.1:2181,127.0.0.1:2182,127.0.0.1:2183/solr"
```

Vérification :

```
http://localhost:8983/solr/#/~cloud
```

```
curl -f "http://localhost:$SOLR_PORT2/solr/admin/info/system?wt=json" | jq '.mode,.lucene'
```

Création de collection :

```
/opt/solr/bin/solr create_collection \
```

```
-c testcoll -shards 2 -replicationFactor 2 \
```

```
-z "127.0.0.1:2181,127.0.0.1:2182,127.0.0.1:2183/solr"
```